```java
public class TreeUtil {

    // 1. root left right
    public static void printPreOrder(BinNode<Integer> bt) {
        if (bt != null) {
            System.out.print(bt.getValue() + "  ");
            printPreOrder(bt.getLeft());
            printPreOrder(bt.getRight());
        }
    }

    // 2. left root right
    public static void printInOrder(BinNode<Integer> bt) {
        if (bt != null) {
            printInOrder(bt.getLeft());
            System.out.print(bt.getValue() + "  ");
            printInOrder(bt.getRight());
        }
    }

    // 3. left right root
    public static void printPostOrder(BinNode<Integer> bt) {
        if (bt != null) {
            printPostOrder(bt.getLeft());
            printPostOrder(bt.getRight());
            System.out.print(bt.getValue() + "  ");
        }
    }

    // 4. assumes bt != null
    public static boolean isLeaf(BinNode<Integer> bt) {
        return bt.getLeft() == null && bt.getRight() == null;
    }

    // 5.
    public static int sumTree(BinNode<Integer> bt) {
        if (bt == null)
            return 0;
        return bt.getValue() + sumTree(bt.getLeft()) + sumTree(bt.getRight());
    }

    // 6.
    public static int countTree(BinNode<Integer> bt) {
        if (bt == null)
            return 0;
        return 1 + countTree(bt.getLeft()) + countTree(bt.getRight());
    }

    // 7.
    public static int sumTreePositive(BinNode<Integer> bt) {
        if (bt == null)
            return 0;
        if (bt.getValue() > 0)
```

```
            return bt.getValue() + sumTreePositive(bt.getLeft()) + sumTreePositive(bt.
    getRight());
        return sumTreePositive(bt.getLeft()) + sumTreePositive(bt.getRight());
    }

    // 8.
    public static int countTreePositive(BinNode<Integer> bt) {
        if (bt == null)
            return 0;
        if (bt.getValue() > 0)
            return 1 + countTreePositive(bt.getLeft()) + countTreePositive(bt.getRight());
        return countTreePositive(bt.getLeft()) + countTreePositive(bt.getRight());
    }

    // 9.
    public static int sumLeftSons(BinNode<Integer> bt) {
        if (bt == null)
            return 0;
        int sum = 0;
        if (bt.getLeft() != null) {
            sum += bt.getLeft().getValue();
        }
        return sum + sumLeftSons(bt.getLeft()) + sumLeftSons(bt.getRight());
    }

    // 10.
    public static int countLeftSons(BinNode<Integer> bt) {
        if (bt == null)
            return 0;
        int count = 0;
        if (bt.getLeft() != null) {
            count = 1;
        }
        return count + countLeftSons(bt.getLeft()) + countLeftSons(bt.getRight());
    }

    // 11.
    public static boolean isExist(BinNode<Integer> bt, int x) {
        if (bt == null)
            return false;
        if (bt.getValue() == x)
            return true;
        return isExist(bt.getLeft(), x) || isExist(bt.getRight(), x);
    }

    // 12.
    public static int countLeafs(BinNode<Integer> bt) {
        if (bt == null)
            return 0;
        if (isLeaf(bt))
            return 1;
        return countLeafs(bt.getLeft()) + countLeafs(bt.getRight());
    }
```

```java
    // 13.
    public static int heightTree(BinNode<Integer> bt) {
        if (bt == null) return -1;
        if (isLeaf(bt)) return 0;
        return 1 + Math.max(heightTree(bt.getLeft()), heightTree(bt.getRight()));
    }

    // 14.
    public static int maxTree(BinNode<Integer> bt) {
        // Base case: return a very small number so it doesn't mess up the max calculation
        if (bt == null) {
            return Integer.MIN_VALUE;
        }
        int maxLeft = maxTree(bt.getLeft());
        int maxRight = maxTree(bt.getRight());
        return Math.max(bt.getValue(), Math.max(maxLeft, maxRight));
    }
}
```